

Clean Code

Code is read far more often than it is written. Clean code respects the reader.

From Structure to Style

Cohesion and coupling shape how the pieces fit. Clean code is about the pieces themselves — how a single class, method, or line reads.



Simplicity

Is it as simple as it can be —
no simpler?



Correctness

Does it do the right thing,
provably?



Completeness

Does it handle everything it
should?

Learning Objectives

- Explain why readability is an economic decision, not a style preference.
- Apply a simple framework: simplicity, correctness, completeness.
- Write intention-revealing names, small functions, and honest error handling.
- Use AI to generate code — and hold it to the same clean-code bar.

You Read Code 10× More Than You Write It

Every hour you save the next reader — often future-you — is paid back many times over. Clean code is how you make the expensive part (reading) cheap.

Dirty code costs

- Slow to understand → slow to change
- Bugs hide in the noise
- Everyone fears touching it

Clean code pays

- Reads like well-written prose
- Intent is obvious; bugs stand out
- Cheap and safe to evolve

```
graph LR; A[Clean Code] --- B[Simplicity]; A --- C[Correctness]; A --- D[Completeness];
```

Clean Code

Simplicity

Correctness

Completeness

Three Questions for Every Piece of Code

Don't memorize a hundred rules. Judge each unit of code against **three** — in order.

01



Simplicity

Is it as simple as it can be — and no simpler? Does it read the way the reader expects?

Can I follow it?

02



Correctness

Does it do the right thing — provably, and within the real system around it?

Is it right?

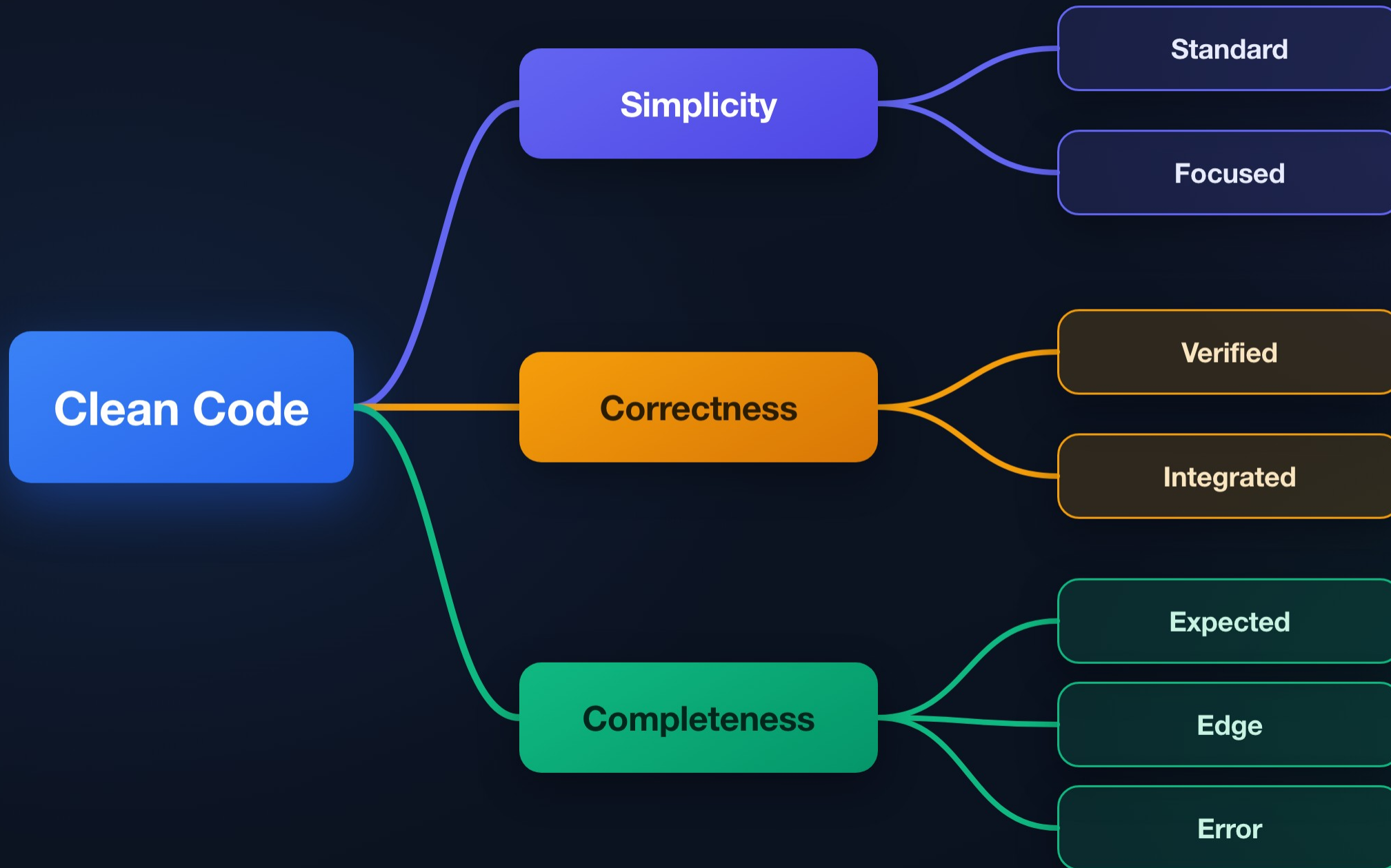
03



Completeness

Does it handle everything it should — the expected, the edge, and the errors?

Is it finished?



Inside the Three Questions

Each pillar splits into concrete checks you can apply to **any** class, method, or line.

Simplicity

Can I follow it?

Correctness

Is it right?

Completeness

Is it finished?

● Standard

Idiomatic and conventional — **familiar formatting**, fits the architecture, no surprises.

● Verified

Proven right — **tests and local reasoning** show it does the right thing.

● Expected

SHOULD HAPPEN

The **happy path** — the intended behavior, handled correctly.

● Edge

MIGHT HAPPEN

Boundaries and empties — the rare-but-valid cases, covered.

● Focused

One job per unit — **small, single-purpose**, and free of duplication.

● Integrated

Right in context — **honors its contracts** and behaves in the real system.

● Error

SHOULDN'T HAPPEN

Bad input and failures — handled honestly; resources cleaned up.

Section 01

Simplicity

The simplest code that clearly says what it means.

Simple Means ‘One Fold’

‘Simple’ is not a vague compliment — it has a precise, ancient meaning. From the Latin simplex: the root sm- (‘one’) folded with -ply / -plex (‘fold’). Literally, one fold.



simplex

sim- (one) + -plex (fold) — one fold. One concern, folded once.



duplex

duo (two) + -plex (fold) — two folds, sitting side by side.



complex

com- (together) + plectere (weave) — folds braided so you can’t pull them apart.

So ‘simple’ never meant easy or small — it meant one fold. The same -ply / -plex fold-thread runs through ply, apply, pliable, plait, complicate.

Turkish, too: *pile* (pileli etek = ‘katlı’) ← French *pli* ← Latin *plicare* — the very same fold.

Names Are Documentation

Use intention-revealing names — *a good name removes the need for a comment. Reveal intent; drop the noise:*

Unclear

```
int d; // elapsed days
List<int> l;
if (u.a && u.b) {
    proc(u);
}
```

Intention-revealing

```
int elapsedDays;
List<int> activeUserIds;
if (user.isActive &&
    user.hasConsent) {
    activate(user);
}
```

Same behavior. The right names turn a puzzle into a sentence.

 **Clean Code, Ch. 2 “Meaningful Names” — Robert C. Martin** — *the “Use Intention-Revealing Names” section this slide is built on*

Naming Is Design

The most powerful design and refactoring tool is not a pattern — it is naming. And not good naming: the act of naming. To name a thing is to make it visible, thinkable, and talkable — in life and in code alike.

Name it

- Visible — you can point at it
- Talkable — you can discuss it
- Testable, documentable, reusable

Leave it unnamed

- Invisible — lost in a bigger whole
- No handle to grab or refer to
- Can't test, document, or reason about

No Name, No Handle

A block of logic with no name is invisible — you can't call it, test it, or even point at it. It exists only inside something bigger.

Unnamed — a block in a method

```
void checkout(Cart cart) {  
    // ... other work ...  
    double t = 0;  
    for (var i : cart.items())  
        t += i.price() * i.qty();  
    if (t > 100) t *= 0.9;    // <- no name  
    // ... more work ...  
}
```

Named — extracted method

```
void checkout(Cart cart) {  
    // ... other work ...  
    double total = discounted(cart);  
    // ... more work ...  
}  
  
double discounted(Cart cart) { ... }  
// now it has a name: testable,  
// callable, and talkable
```

Extraction is naming. The moment that block becomes `discounted()`, it is a thing you can test, document, reuse — and discuss.

Extract, Don't Add

New functionality tempts you to add — more lines to a method, more methods to a class. Instead, extract it into a new named abstraction.

Add — grows and hides

- New lines inside an existing method
- New methods on an already-fat class
- The class does more, means less

Extract — names and focuses

- Pull it into a new method or class
- The new name gives it an identity
- What's left is more single-minded

Software grows by extracting, not just adding functionality. Perfection is achieved by extracting — and naming is how you reach functional cohesion.

Primitive Obsession → Value Objects

Primitive obsession — reaching for String and double where a real type belongs. The class hides concepts; group the fields that belong together and name the group as a value object.

Flat primitives

```
class Employee {  
    String name;  
    String street;  
    String city;  
    String zipcode;  
    double salary;  
    double educationPayment;  
    double disabilityPayment;  
}
```

Named value objects

```
class Employee {  
    String name;  
    Address address;  
    Compensation compensation;  
}  
record Address(  
    String street, String city, String zipcode) {}  
record Compensation(  
    double salary, double educationPayment,  
    double disabilityPayment) {}
```

Address and Compensation are now things. You can validate a zip code, total a package, test and reuse them — because the cluster finally has a name.

Reuse Reveals a Name

You wrote a method that felt atomic — nothing to pull apart. Then a second place needs some of its lines. Don't copy them — extract them into a new, well-named method both callers share.

Copy-paste — duplication

```
void placeOrder(Order o) {  
    ...  
    a.setCity(upper(a.city()));  
    a.setZip(digits(a.zip()));  
    ...  
}  
void importOrder(Order o) {  
    ...  
    a.setCity(upper(a.city())); // copied  
    a.setZip(digits(a.zip())); // copied  
    ...  
}
```

Extract & name — reuse

```
void placeOrder(Order o) {  
    ...  
    normalize(o.address());  
    ...  
}  
void importOrder(Order o) {  
    ...  
    normalize(o.address());  
    ...  
}  
void normalize(Address a) { // a new name  
    a.setCity(upper(a.city()));  
    a.setZip(digits(a.zip()));  
}
```

Reuse doesn't mean copy — it means finding a name.

Don't Copy — Cut

The moment your hand reaches for copy-and-paste, stop. What you actually want is cut-and-paste — into something new that deserves a name.

Copying part of a class?

Cut it into a new class with a name that says what it does — then rename the old one: it now does less than its name claims.

Copying part of a method?

The same move: cut it into a new method with a good name. The old method has changed shape too, so its name probably must change as well.

As the software grows you end up with smaller, better-named abstractions — instead of larger, vaguer ones.

Cut when the two copies are the same idea. *If they merely look alike, that is false DRY — give them two honest names instead.*

Small Functions, One Job

A function should do one thing, at one level of abstraction, and be short enough to hold in your head all at once.



Prefer

- Short functions with clear names
- One level of abstraction each
- Few parameters (ideally 0–2)



Avoid

- Long methods that scroll off-screen
- Mixing high- and low-level detail
- Boolean flags that hide two behaviors

Flatten Nested if's

Deeply nested if's — 'if-hell' — flatten into guard clauses, then give the whole decision a name.

if-hell — nested

```
if (user != null) {  
  if (user.isActive()) {  
    if (order.total() > 0) {  
      if (inStock(order)) {  
        ship(order); // buried  
      }  
    }  
  }  
}
```

guard clauses — flat

```
// guards first  
if (user == null)      return;  
if (!user.isActive())  return;  
if (order.total() <= 0) return;  
if (!inStock(order))   return;  
// then the action  
ship(order);
```

named guard — one word

```
if (!canShip(user, order))  
  return;  
ship(order);  
boolean canShip(User u,  
                 Order o) {  
  return u != null  
    && u.isActive()  
    && o.total() > 0  
    && inStock(o);  
}
```

Then name the whole decision. Those guards are one atomic behavior — 'can we ship?'. Extracted as `canShip()`, it becomes a thing you can reuse, test, and talk about — the naming principle, again.

Don't Repeat Yourself

Every piece of knowledge should have one home. Duplication means every change must be made in several places — and one will be forgotten.

- Duplicated logic → duplicated bugs and inconsistent fixes.
- Extract the shared idea into one well-named function or type.
- But beware false DRY: code that looks alike yet changes for different reasons.

DRY in Practice

Same code, twice — but is 10% vs 15% an accidental drift, or two real rules for two contexts? The answer picks the fix.

Duplicated

```
// order context
d = o.total() > 100
  ? 0.10 : 0.0;
// invoice context
d = i.total() > 100
  ? 0.15 : 0.0;
```

One rule, one home

```
double rate(double t) {
  return t > 100
    ? 0.10 : 0.0;
}
// order & invoice
// call ONE rule –
// it cannot drift
```

Two rules, two names

```
double orderRate(double t) {
  return t > 100 ? 0.10 : 0;
}
double invoiceRate(double t){
  return t > 100 ? 0.15 : 0;
}
// each rule named
```

Bug or decision? Drift by accident → give the rule one home. Two genuinely different rules → two honest names (merging would be false DRY).

You can only be DRY by naming — the one way to remove duplication is to extract the shared idea and name it (our naming principle again).

Share the Shape, Name the Policy

Can one method serve both? Yes — but only if the parameter carries a value, not a hidden choice.

Merged with a flag

```
double discountRate(
    double t, boolean invoice) {
    double pct = invoice ? 0.15 : 0.10;
    return t > 100 ? pct : 0.0;
}
// call: discountRate(total, true)
//      ^ true means... what?
```

Shared shape, named policies

```
double tieredRate(double t, double pct){
    return t > 100 ? pct : 0.0;
}
double orderRate(double t) {
    return tieredRate(t, 0.10);
}
double invoiceRate(double t) {
    return tieredRate(t, 0.15);
}
```

A value, not a flag. `tieredRate` holds the shared shape once; `orderRate` / `invoiceRate` keep honest names. A boolean selector would just be a mystery argument at every call site.

When rates are open-ended — per-customer, per-region, a pricing table — pass the value or look it up; named wrappers stop scaling.

Comment the 'Why', Not the 'What'

If the code needs a comment to explain what it does, first try to make the code say it:

Java

Good comments explain intent and trade-offs

```
// BAD: restates the code
```

```
i++; // increment i
```

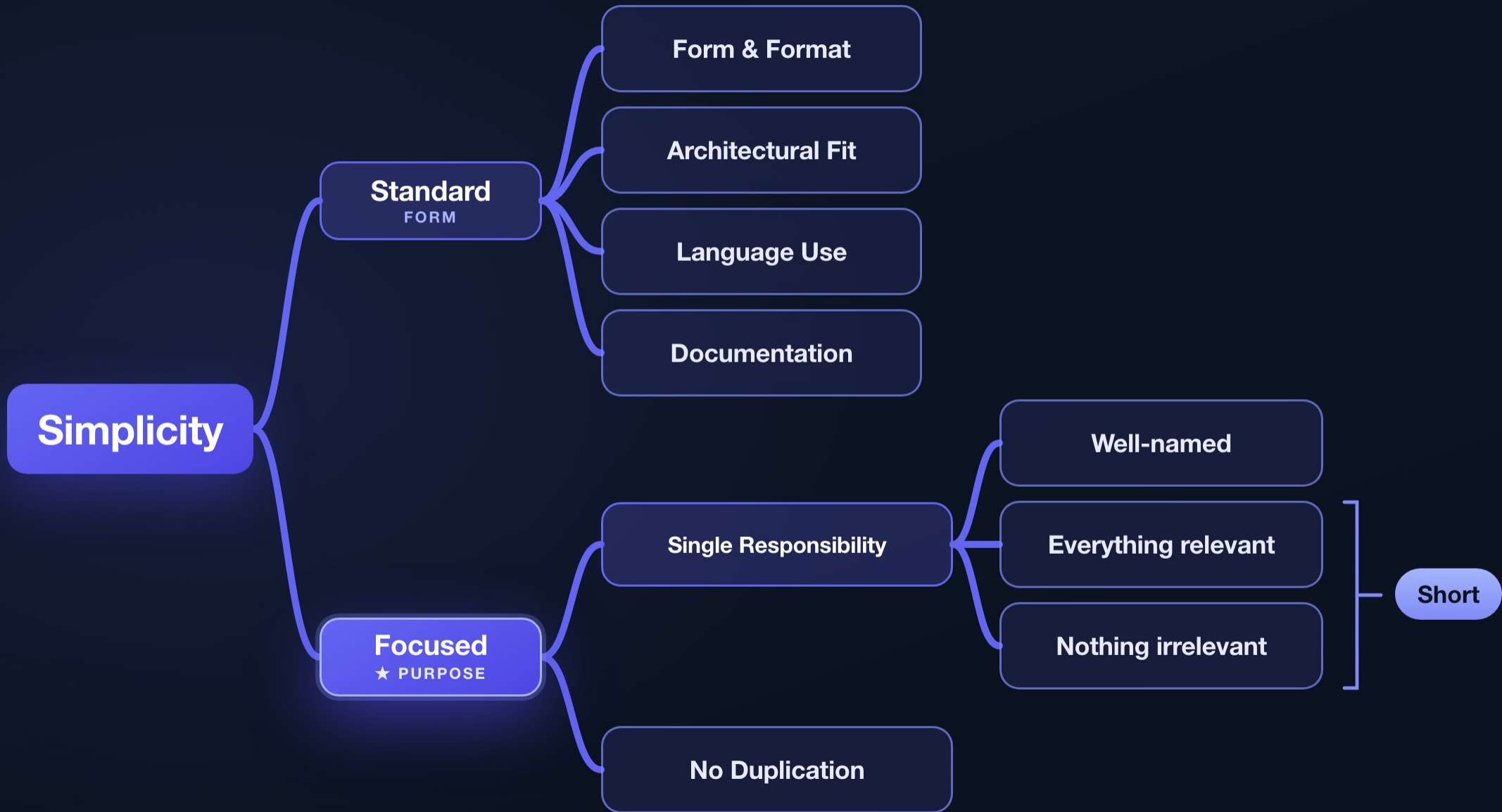
```
// GOOD: explains a decision the code can't
```

```
// Retry once: the payment gateway drops the first call after a cold start.
```

```
retryOnce(() -> gateway.charge(order));
```

The best comment is a good name. The next best explains a 'why' the code can't.

 **Clean Code, Ch. 4 “Comments” — Robert C. Martin** — *the definitive chapter on documentation — when a comment earns its place*



Form **vs** Purpose

Simplicity asks two different things of your brain —
perceive the code, then **abstract** it.

STANDARD

PERCEIVE

Form

The **surface** that shapes your first impression — before you know what the code means.

- › Matches conventions — **nothing surprising** to slow the eye
- › Works by **recognition** — fast and automatic
- › Largely **automatable** — formatters, linters, AI

Like typography — it makes the code easy to read.

Easy to see

FOCUSED

ABSTRACT

★ Purpose

The **one job** your brain can name, box away, and trust — without reading the insides.

- › One unit → **one chunk** of working memory
- › **Fewer things to hold** — trust the box, move on
- › Needs **judgment** — no tool finds the seams

Like chapters — it makes the ideas easy to follow.

Easy to trust — and move on

Standard makes code easy to **see** · Focused makes it easy to **forget** AI writes **Form** for free — your review is about **Purpose**: is this really one job?

Inside Simplicity



Section 02

Correctness

Clean code that does the wrong thing is just tidy failure.

It Must Do the Right Thing

Readability serves correctness — not the other way round. Clean code makes correctness visible: you can see that it is right.

- Make the expected behavior obvious from the code's shape.
- Prefer code whose correctness you can reason about locally.
- Tests pin correctness down — we devote a whole session to them soon.

Handle Errors Honestly

Don't hide failure behind magic values or empty catches. Fail fast, fail loud, fail clearly:

Hides failure

```
User find(int id) {  
    try {  
        return db.get(id);  
    } catch (Exception e) {  
        return null; // swallowed  
    }  
}
```

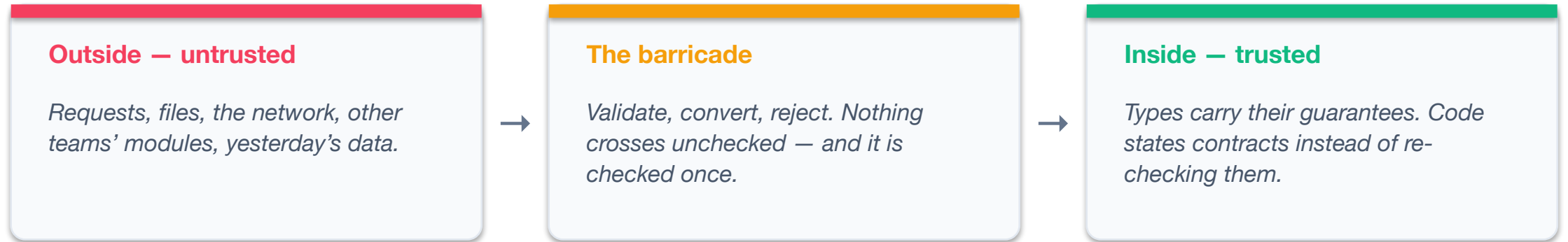
Honest

```
User find(int id) {  
    User u = db.get(id);  
    if (u == null)  
        throw new NotFound(id);  
    return u;  
}
```

A swallowed exception turns a clear failure into a mysterious one later.

Defensive Programming

You have been doing this since Simplicity — guard clauses, honest failures. The name for that stance is defensive programming: assume the caller can be wrong, and make the mistake stop here instead of spreading.



Validate at the barricade, then trust. A check repeated deep inside is either duplicated work — or a sign the barricade is in the wrong place.

📖 **Code Complete, 2nd ed., Ch. 8 “Defensive Programming” — Steve McConnell** — *barricades, assertions, and how much defense to leave in production*

Defend, Contract, or Assert?

‘Always be defensive’ is bad advice. There are three stances, and the skill is choosing the right one for the place:

Stance	It says	Where it belongs
Defensive	Trust nothing that comes in; check it	At trust boundaries: input, I/O, other systems
By contract (Meyer)	State who guarantees what — don’t re-check	Inside the barricade, between your own modules
Offensive	Crash loudly the moment an assumption breaks	Assertions for ‘can’t happen’, while developing

Too little — a bad value travels far, and corrupts data before anyone notices.

Too much — every method re-checks the same rule, the copies drift, and real failures get swallowed ‘to be safe’.

Defend at the boundary · contract on the inside · assert while you build.

The Principle of Least Astonishment

Code should behave the way its name and shape promise. Surprises are where bugs and misuse live.



Astonishing

- `getUser()` also writes to the database
- A 'safe' method throws on valid input
- Names that lie about what they do



Predictable

- Queries read; commands change
- Behavior matches the name
- No hidden side effects

Astonishment in the Wild

The name promises a read. The body writes. Every caller is now wrong about their own code.

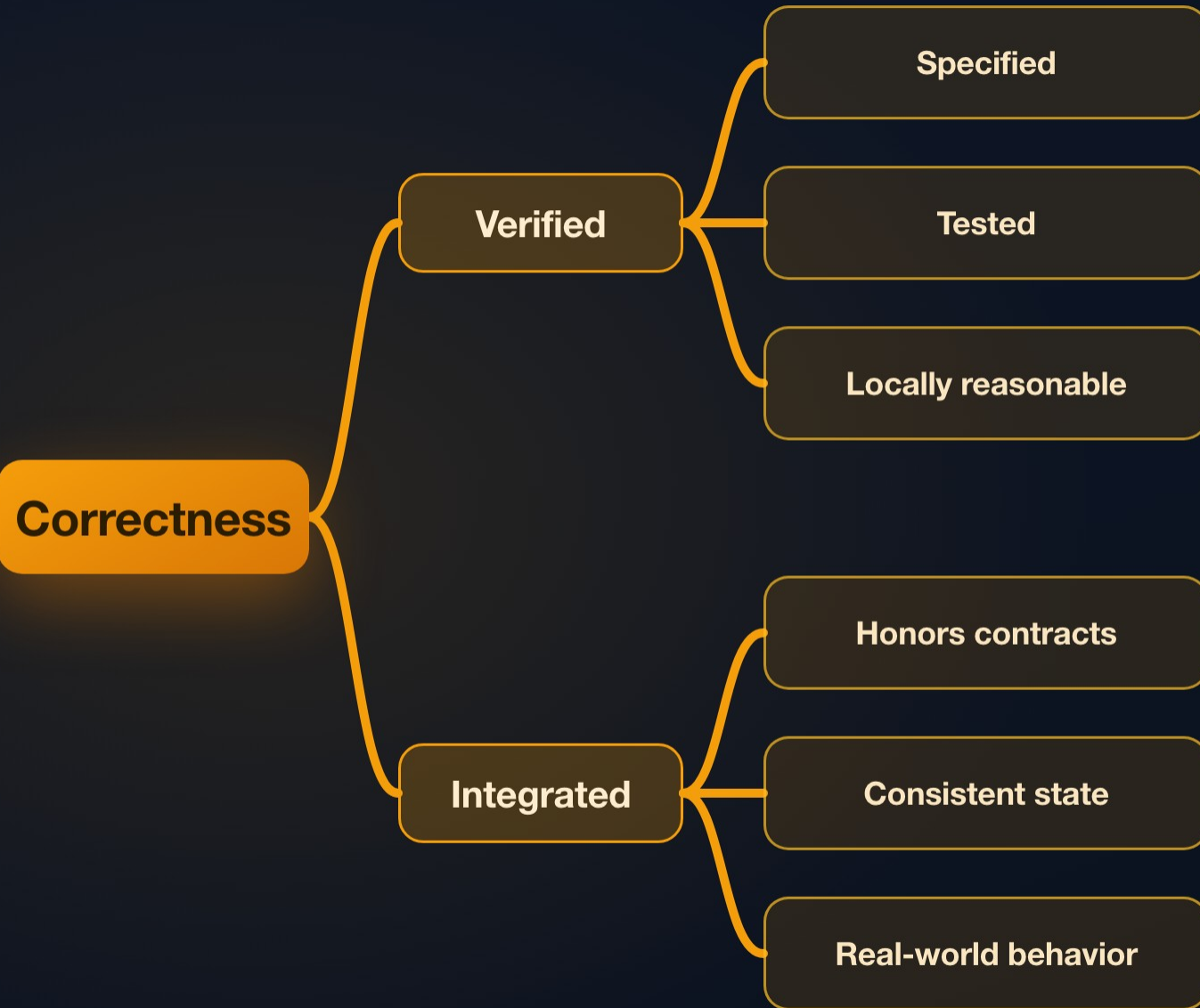
The name lies

```
User getUser(int id) {  
    User u = db.find(id);  
    u.setLastSeen(now()); // surprise:  
    db.update(u);          // a 'get'  
    return u;              // that writes  
}
```

Query and command, split

```
User findUser(int id) {  
    return db.find(id); // reads only  
}  
  
void recordVisit(User u) {  
    u.setLastSeen(now()); // the write,  
    db.update(u);         // named honestly  
}
```

Call `getUser()` in a loop of 1,000 users and you just wrote to the database 1,000 times — without a clue. Queries read; commands change; names tell the truth.



Inside Correctness

Correctness

Verified

Right on its own

Specified

The expected behavior is defined

Tested

Automated tests demonstrate it holds

Locally reasonable

You can see it's right by reading it

Integrated

Right within the whole

Honors contracts

Respects what callers & collaborators expect

Consistent state

Leaves data valid — no half-finished changes

Real-world behavior

Holds up with real data, timing & failures

THE PAYOFF

You can *see* that it's *right*.

Correctness you can read — provable, not hopeful.

Section 03

Completeness

The 80% that works is not the same as done.

Cover the Edges

Happy-path code is a demo, not a feature. Completeness means the awkward inputs are handled on purpose, not by accident.

- Empty, null, zero, negative, and huge inputs — decide what each does.
- Boundaries: first and last element, off-by-one, overflow.
- Concurrency and ordering, where they apply.
- Make the decision explicit in code — don't leave it to chance.

The Edges in Practice

This passes the demo. Two edge cases are hiding in four lines — one crashes, one is silently wrong.

Works in the demo

```
double average(List<Integer> xs) {  
    int sum = 0;  
    for (int x : xs) sum += x;  
    return sum / xs.size();  
}
```

```
// []      -> ArithmeticException  
// [1, 2]  -> 1.0  (int division!)
```

Edges decided on purpose

```
OptionalDouble average(  
    List<Integer> xs) {  
    return xs.stream()  
        .mapToInt(Integer::intValue)  
        .average();  
}
```

```
// []      -> OptionalDouble.empty()  
// [1, 2]  -> 1.5
```

The return type is the decision. OptionalDouble forces every caller to say what an empty average means — the edge case can no longer be forgotten.

Clean Up After Yourself

Files, connections, locks — whatever you open, you must close, even when something throws:

Java

```
try (var in =  
    open(path)) {  
    return read(in);  
} // auto-closed
```

C#

```
using var in =  
    Open(path);  
return Read(in);  
// auto-closed
```

Python

```
with open(path)  
    as f:  
    return read(f)  
# auto-closed
```

Language idioms (try-with-resources, using, with) make cleanup automatic — use them. More on this in Secure Coding.

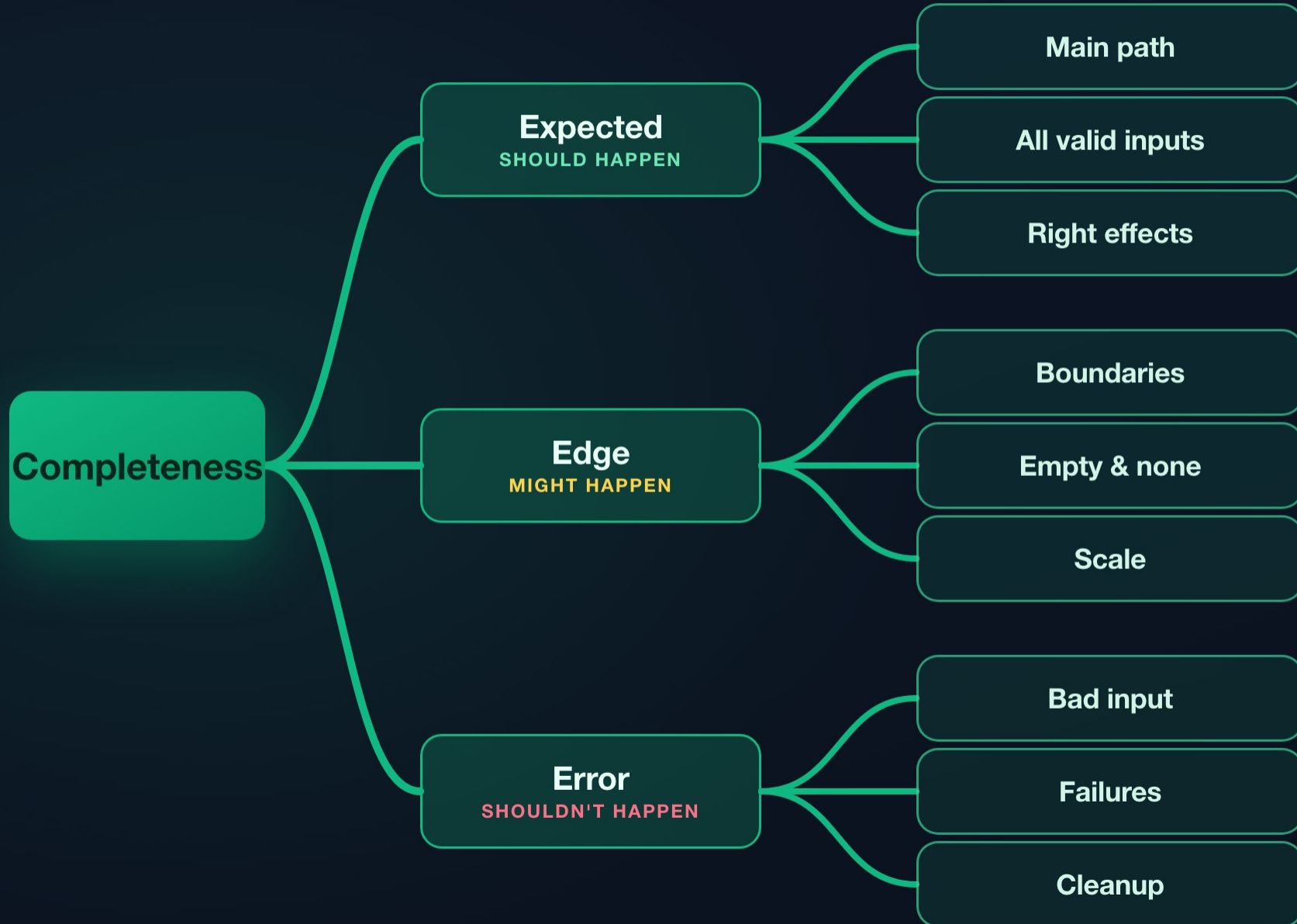
Finished Means Consistent

Signs of 'not done'

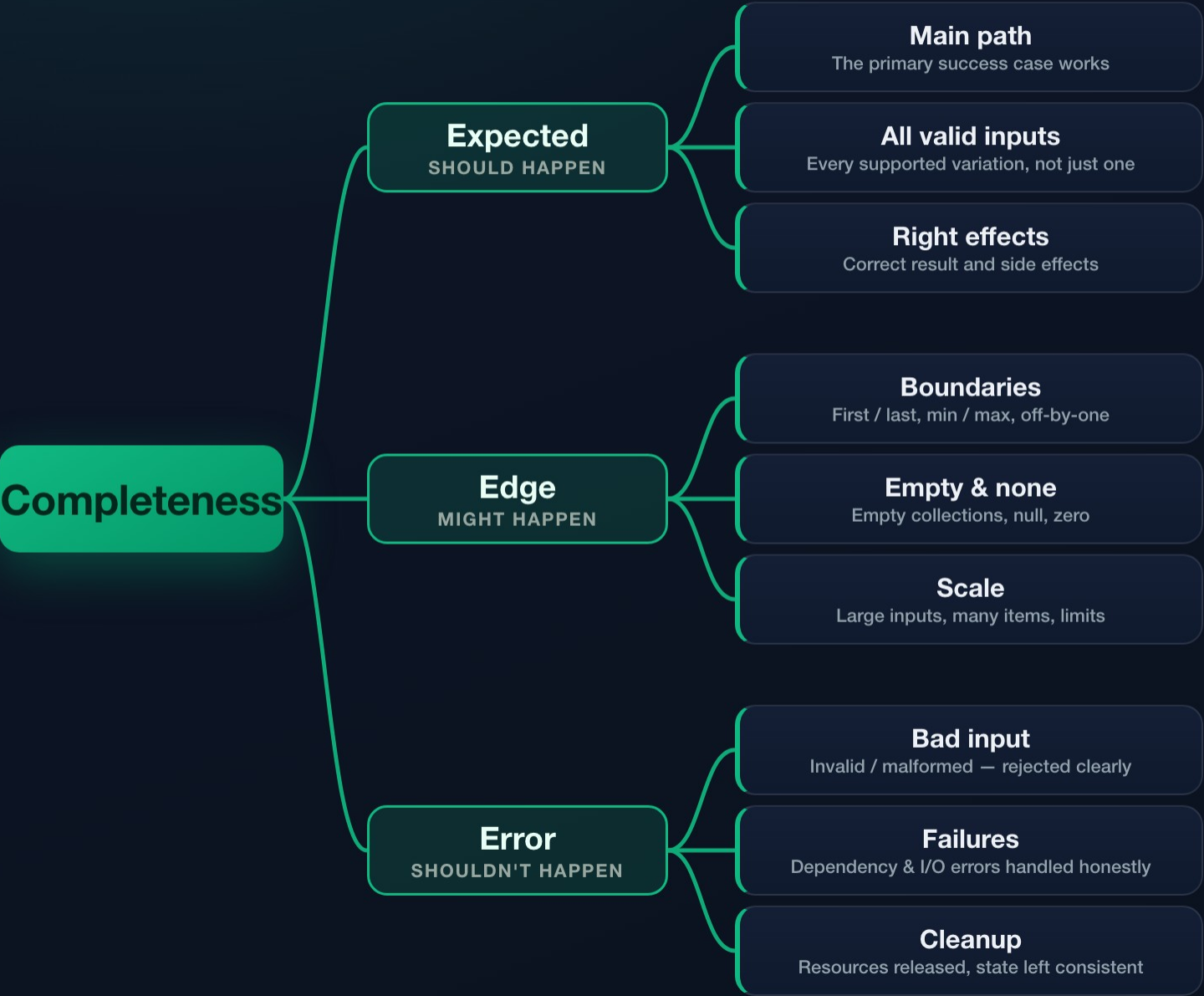
- TODOs and commented-out code
- Mixed styles and naming
- Dead code no one calls

Signs of finished

- One consistent style throughout
- No loose ends or leftovers
- Reads as if written by one hand



Inside Completeness



THE PAYOFF

Nothing left to surprise you.

Every path — expected, edge, and error — accounted for.

Clean Code Serves the Two Forces

Clean code isn't a separate goal — it's how cohesion and coupling show up line by line.



Good names & small units

→ higher cohesion: each piece has one clear job.



Honest interfaces, no surprises

→ lower coupling: callers depend on intent, not internals.



Completeness & cleanup

→ changeability: fewer nasty surprises when things evolve.

AI Writes Fast — You Keep It Clean

AI will happily generate working-but-messy code. Your job shifts from typing to judging: does this meet the bar?

- Review AI output for names, size, duplication, and honest errors.
- Ask for the simplest version — then ask it to justify any complexity.
- Encode your standards in the prompt; make 'clean' the default, not a rewrite.

Works ≠ clean

AI's First Draft

Here is the kind of code an AI — me included — will hand you: it compiles and passes the test, but it is stringly-typed, cryptically named, and over-commented.

AI's draft — works, but messy

```
double calcTotalPrice(
    List<Map<String,Object>> d) {
    double total = 0.0;        // init to zero
    for (var item : d) {
        double p = (double) item.get("price");
        int q = (int) item.get("quantity");
        total = total + (p * q); // add it up
    }
    return total;              // the result
}
```

What you keep

```
double total(List<LineItem> items) {
    return items.stream()
        .mapToDouble(LineItem::subtotal)
        .sum();
}
```

Same result, a third of the code. AI gets you to 'works'; you get it to 'clean' — review for real types, honest names, and comments that don't just restate the code.

One Standard, Three Questions

Judge every piece of code against three questions — **in order**.

Can I follow it?

Simplicity

Standard & Focused — idiomatic, one job, no duplication.

→ *Easy to understand, easy to change.*

Is it right?

Correctness

Verified & Integrated — proven on its own and in the system.

→ *You can see that it's right.*

Is it finished?

Completeness

Expected · Edge · Error — should, might, and shouldn't happen.

→ *Nothing left unhandled.*

Simple → **Right** → **Complete** One bar for every piece of code — and hold AI-written code to the same standard.

The Code Behind This Session

Every example in this session compiles and runs — the same lessons in three languages, with the smell and the fix sitting side by side:

Java

Maven · JUnit 5 · 3 tests here

`cleancode/guardclauses/`
EligibilitySmell → flat guard clauses

`cleancode/honesterrors/`
UserFinderSmell → throws instead of null

C# / .NET

xUnit · 2 tests here

`CleanCode.cs`
guard clauses and honest errors

`CleanCodeTests.cs`
same behavior, honest failure

Python

pytest · 2 tests here

`cleancode.py`
guard clauses and honest errors

`tests/test_cleancode.py`
raises instead of returning None

github.com/kaldiruglu/Agentic-Software-Design-and-Architecture-Bootcamp-JAVA · [-DOTNET](#) · [-PYTHON](#)

The smell still passes its test. That is the point: clean code is not about behavior — it is about the reader, and only the reader can tell the difference.

*Clone one, run the suite, then break something and watch which test goes red. **Across all chapters the suites are 70 · 63 · 65 tests.***

Remember these

Key Takeaways

- ✓ Code is read far more than written — optimize for the reader.
- ✓ Judge every piece by three questions: simple, correct, complete.
- ✓ Names are documentation; small functions do one thing.
- ✓ Handle errors honestly and surprise no one.
- ✓ With AI, you are the reviewer who holds the clean-code bar.

Go deeper

Further Reading

The books behind this session. If you read only one, read the first.

📖 **Clean Code** Robert C. Martin · Prentice Hall, 2008

Ch. 2 Meaningful Names — “use intention-revealing names”; Ch. 4 Comments — the best chapter written on documentation.

📖 **Code Complete, 2nd ed.** Steve McConnell · Microsoft Press, 2004

Ch. 11 The Power of Variable Names; Ch. 32 Self-Documenting Code — broader, evidence-based.

📖 **Five Lines of Code** Christian Clausen · Manning, 2021

Mechanical refactoring rules instead of smells you must “feel” — sharpens the simplicity checks.

📖 **Clean Code Cookbook** Maximiliano Contieri · O’Reilly, 2023

One recipe per smell — a lookup table for the fixes we practice in this session.

Online — code smells & refactorings, illustrated step by step → refactoring.guru

What's Next

Module 4

SOLID

with Akin Kaldıroğlu

Five principles that turn 'high cohesion, low coupling' into concrete, repeatable design moves.

Hunt the Smells, Then Clean Up

Catalog what's wrong with the ACME Orders code, then make it simple, correct and complete.

Refactoring course · Smells & Clean Code

Repo github.com/kaldiroglu/refactoring-course-student-materials

IN THIS SESSION, LOOK AT

- ▶ Exercises/Ex02-SmellInventory.md
- ▶ .../util/Util.java , .../service/PricingHelper.java
- ▶ Slides-PDF/Ch04-CodeSmells.pdf

Questions?

Simple. Correct. Complete.